

Day 3 Instructor Run Sheet (120 minutes)

For the instructor only. Teach in Hindi. All written material stays in English. Day 3 has a new risk: 30 to 60 computers download Express with `npm install` at the same time. Plan for it.

Before the class (do these first)

Do **Pilot B** in `INSTRUCTOR-PILOT-CHECKLIST.md` on a lab PC: `npm init -y`, then `npm install express`. Does it work on the lab network? Write down the time: _____ (not measured yet). If you can, ask 5 to 10 computers to run `npm install express` at the same time (Pilot B2). Write down what happens: _____ (not measured yet). Prepare the **fallback**: a zipped `node_modules` folder (see "Risk: npm install" below). Put the zip on the LAN share and on two spare USB sticks. Put the Day 3 `starter` and `done` folders on the same share. The starter must be a copy of the Day 2 `done` folder. Run the `done` project yourself once: `npm install`, `copy .env.example .env`, `node server.js`, open `http://localhost:3000`. Check which Windows window appears when a Node.js server starts on a lab PC (a firewall window may appear). Write down what students should do: _____ (not tested). Check that the lab browser's Console accepts pasted code, or asks for `allow pasting` (Part 7). Write down what you saw: _____ (not tested). Zoom VS Code and the terminal (**Ctrl and +**) so that the back row can read. Day 2 follow-up: check first the students who could not install Node.js on Day 2 (your list from Day 2).

Timeline

Minutes	What	Notes
0 to 8	Open: Day 2 questions, homework, GitHub accounts. Ask: "What happened to your notes when you refreshed?"	Students run <code>node -v</code> . Part 0 of the lab is done in this time for those who are ready
8 to 16	Talk: the client and the server, what an API is, GET, POST, DELETE, status codes, JSON	Draw the picture from the Theory (browser, server, <code>data.js</code>). Show the table of three routes. Do not teach every status code: 200, 201, 204, 400, 401, 404 only
16 to 26	Lab Parts 0 and 1: check the setup, make the <code>public</code> folder	Drag and drop in the VS Code file list. Walk the room: every student shows the folder tree
26 to 40	Lab Part 2: <code>npm init -y</code> and <code>npm install express</code>	Talk 3 minutes: package, npm registry, <code>package.json</code> , <code>node_modules</code> . Gate: most students must have <code>npm ls express</code> working before you go on. If not, use the fallback (see below). Use pods of six: the first student who is done helps the others
40 to 50	Lab Part 3: first Express server	Type <code>server.js</code> live. Explain <code>require</code> , <code>app</code> , <code>express.static</code> , <code>listen</code> . Show <code>Cannot GET /nothing</code>
50 to 60	Lab Part 4: <code>data.js</code>	Talk 3 minutes: <code>async</code> and <code>await</code> as "wait for the answer". Do not go deep. Say: "On Day 4 these three functions will use a database"
60 to 68	Lab Part 5: the GET route	Open <code>/api/notes</code> in the browser, then show the Network tab: status 200, method GET
68 to 80	Lab Part 6: password, <code>.env</code> , <code>.gitignore</code>	Talk 4 minutes: what is a secret, why not in the code, why not in Git. Warn about <code>.env.txt</code> from Notepad. Do the Part 6e demo once on the projector
80 to 92	Lab Part 7: POST, DELETE and <code>requirePassword</code>	Explain middleware with the gate guard picture. Test with the Console helper. Show 401, 400, 201, 404, 204. Say: "The check is on the server, so the page cannot cheat"
92 to 110	Lab Part 8: password box and the <code>fetch</code> version of <code>app.js</code>	Walk through sections 1 to 9. Point out what changed from Day 2: <code>renderNotes(notes)</code> receives the list, and <code>loadNotes</code> , <code>addNote</code> and <code>deleteNote</code> use <code>fetch</code>

Minutes	What	Notes
110 to 117	Lab Part 9: tests	Do all tests. The last one (restart the server) is the main point: notes are lost. Say: "This is why we need a database tomorrow"
117 to 120	Quiz or homework, backup	

Cut lines (if the class runs late)

Parts 2, 3, 6, 7 and 8 are the priority.

1. **Quiz:** move to the start of Day 4. This is the first thing to cut.
2. **Part 5 (Network tab):** show it on the projector only.
3. **Part 6e (see the safety check work):** instructor demo only.
4. **Part 7a (Console tests):** instructor demo only. Students test with the page in Part 9.
5. **Part 4 (data.js):** students copy `data.js` from the `done` folder, and you explain it in 3 minutes.
6. **Part 8b (app.js):** students copy `public\js\app.js` from the `done` folder. You explain the three `fetch` calls (sections 4, 5 and 6). Students still add the password box to `index.html` by hand.

Do not cut: a running Express server (Parts 2 and 3), the `.env` and the password check (Parts 6 and 7), the page that talks to the server (Part 8), and the "notes are lost on restart" test (Part 9).

Risk: npm install on 30 to 60 PCs at once

Not yet tested. Nobody has run `npm install express` on the lab network. If many computers download at the same time and the internet is weak, installs may be slow or fail. This is why Pilot B exists.

Fallback plan (not yet tested on a lab PC):

1. On your own computer, make a copy of the Day 3 `done\notes-board` folder. Do not do this inside the Git repository.
2. In the copy, run `npm install`.
3. Make a zip file of the `node_modules` folder (right-click, then **Send to**, then **Compressed (zipped) folder**, or the menu text on your Windows).
4. Put the zip on the LAN share, together with the `package.json` and `package-lock.json` from the `done` folder.

5. A student whose `npm install` fails copies the zip, unzips it **into their own** `notes-board` **folder** so that the path `notes-board\node_modules\express` exists, and copies in the `package.json` from the share.
6. Then the student runs `npm ls express` to check.

What we know and what we do not know:

Item	Status
<code>npm install</code> of the <code>done</code> project works on Linux with Node.js 24.21.0 and gives Express 5.2.1	Observed
The packages in <code>node_modules</code> are pure JavaScript (no <code>.node</code> files and no <code>binding.gyp</code> files were found in the Linux copy)	Observed on Linux
So the same folder should work on Windows	Inferred, not tested
Zippping and unzipping <code>node_modules</code> on a Windows 11 Home lab PC	Not tested
Speed of the LAN share when many students copy the zip together	Not tested

Write down the result of your own test here: _____

If installs fail for some students

- Use the zipped `node_modules` fallback above.
- If a student's computer still fails, pair that student with a neighbour. They can code on the neighbour's computer and fix their own after class.
- Make a list of those students. Check them first on Day 4.

What to say about common problems

- "After you change `server.js`, `data.js` or `.env`, stop the server with Ctrl + C and start it again."
- "The server keeps the terminal busy. If the prompt came back, the server is not running."
- "If you see 'Could not start the server', another server is using port 3000. Find it and stop it with Ctrl + C." (The server code prints this message. Observed on Linux with Express 5.2.1; not tested on Windows.)
- "Create `.env` in VS Code, not in Notepad."
- "Open the page at `http://localhost:3000`. Do not double-click the file any more."
- "A red message is a clue. Read it."

Talk points

- **API:** a set of addresses that give data to programs. The page is one user of the API. The Console is another.
- **Methods and codes:** GET reads, POST adds, DELETE removes. 2xx good, 4xx the client made a mistake.
- **Middleware:** a guard at the gate. `requirePassword` is ours.
- **Secrets:** the password lives in `.env`. `.env` lives in `.gitignore`. Never upload it.
- **Server check:** anyone can read and change browser JavaScript. Only the server can protect the password. This is a simple teaching lock (one shared password, no accounts).
- **Safe text:** `textContent`, again. Add `<b>bold</b>` and show it as plain letters.
- **The lesson at the end:** refresh keeps the notes (they are on the server), but a server restart loses them (they are in memory). Tomorrow: a database.

Before ending the class

Every student who could run the server has seen the "restart loses the notes" test.

Students with problems are on your list.

Students know the Day 4 homework: GitHub username, read the Day 4 theory once, and keep their `.env` password in mind.

Backup done. (Students may skip `node_modules`.)

Notes for your own preparation tonight (Day 4)

- The Day 4 `starter` folder must be a copy of the Day 3 `done` folder.
- Pilot C in `INSTRUCTOR-PILOT-CHECKLIST.md` (built-in SQLite on Node.js 24) is still open.